C# PRACTICAL 2

Function Overloading, Inheritance, Interface and Exception Handling

Practical 2A: Function Overloading

Code:

```
using System;

namespace B4_Arjun_Nitin
{
    class area
    {
        public void CalculateArea(int side)
        {
            Console.WriteLine("Area of Square = " + (side * side));
        }

        public void CalculateArea(int length, int breadth)
        {
            Console.WriteLine("Area of Rectangle =" + (breadth * length));
        }

        public double CalculateArea(float p, double rad)
        {
            return (p * rad * rad);
        }
    }

    internal class functionoverloading
    {
        static void Main(string[] args)
        {
            area obj = new area();

            Console.WriteLine("Enter the side");
            int side = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(side);

            Console.WriteLine("Enter the length of Rectangle :");
            int length = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Enter the breadth of Rectangle :");
            int breadth = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(length, breadth);

            Console.WriteLine("Enter the Radius of circle :");
            double rad = double.Parse(Console.ReadLine());
            Console.WriteLine("circle" + obj.CalculateArea(3.14f, rad));
        }
    }
}
```

Output:

```
Enter the side
2
Area of Square = 4
Enter the length of Rectangle :
3
Enter the breadth of Rectangle :
2
Area of Rectangle =6
Enter the Radius of circle :
5
circle78.5000026226044
```

Practical 2B: Inheritance

Code:

```
using System;

namespace B4_Arjun_Nitin
{
    class vehicle
    {
        string name = "Audi";

        public void dispvehicle()
        {
            Console.WriteLine("vehicle name =" + name);
        }
    }

    class car : vehicle
    {
        string model = "2026";

        public void dispcar()
        {
            dispvehicle();
            Console.WriteLine("Car model =" + model);
        }
    }

    class sportscar : car
    {
        int speed = 450;

        public void dispsportscar()
        {
            dispcar();
            Console.WriteLine("Max speed =" + speed);
        }
    }

    class bike : vehicle
    {
        string type = "classic";

        public void dispbike()
        {
            dispvehicle();
            Console.WriteLine("bike type =" + type);
        }
    }

    internal class inheritance
    {
        static void Main(string[] args)
        {
            sportscar s = new sportscar();
            bike b = new bike();

            s.dispsportscar();
            b.dispbike();
        }
    }
}
```

Output:

```
vehicle name =Audi
Car model =2026
Max speed =450
vehicle name =Audi
bike type =classic
```

Practical 2C: Interface

Code:

```
using System;

interface Area
{
    double compute(double x);
}

class square : Area
{
    public double compute(double x)
    {
        return x * x;
    }
}

class circle : Area
{
    public double compute(double x)
    {
        return Math.PI * x * x;
    }
}

class interf
{
    public static void Main(string[] args)
    {
        square sq = new square();
        circle c = new circle();
        Area a;

        a = sq as Area;
        Console.WriteLine("Area of Square=" + a.compute(10.0));

        a = c as Area;
        Console.WriteLine("Area of Circle=" + a.compute(10.0));
    }
}
```

Output:

```
Area of Square=100
Area of Circle=314.159265358979
```

Practical 2D: Exception Handling

Code:

```
using System;

class invalidage : Exception
{
    public invalidage(String msg) : base(msg)
    {
    }
}

class excepthan
{
    static void ValidateAge(int age)
    {
        if (age < 0 || age > 100)
        {
            throw new invalidage("invalid Age");
        }
    }

    public static void Main(string[] args)
    {
        try
        {
            int a = 10, b = 2;
            Console.WriteLine(a / b);
        }
        catch (DivideByZeroException)
        {
            Console.Write("Deniminator should not be zero");
        }
        finally
        {
            Console.Write("In Finally....");
        }

        try
        {
            Console.Write("\nEnter age:");
            int age = int.Parse(Console.ReadLine());
            ValidateAge(age);
        }
        catch (invalidage ex)
        {
            Console.WriteLine("Try with other Input !");
        }
    }
}
```

Output (valid age):

```
5
In Finally....
Enter age:35
```

Output (invalid age):

```
5
In Finally....
Enter age:109
Try with other Input !
```